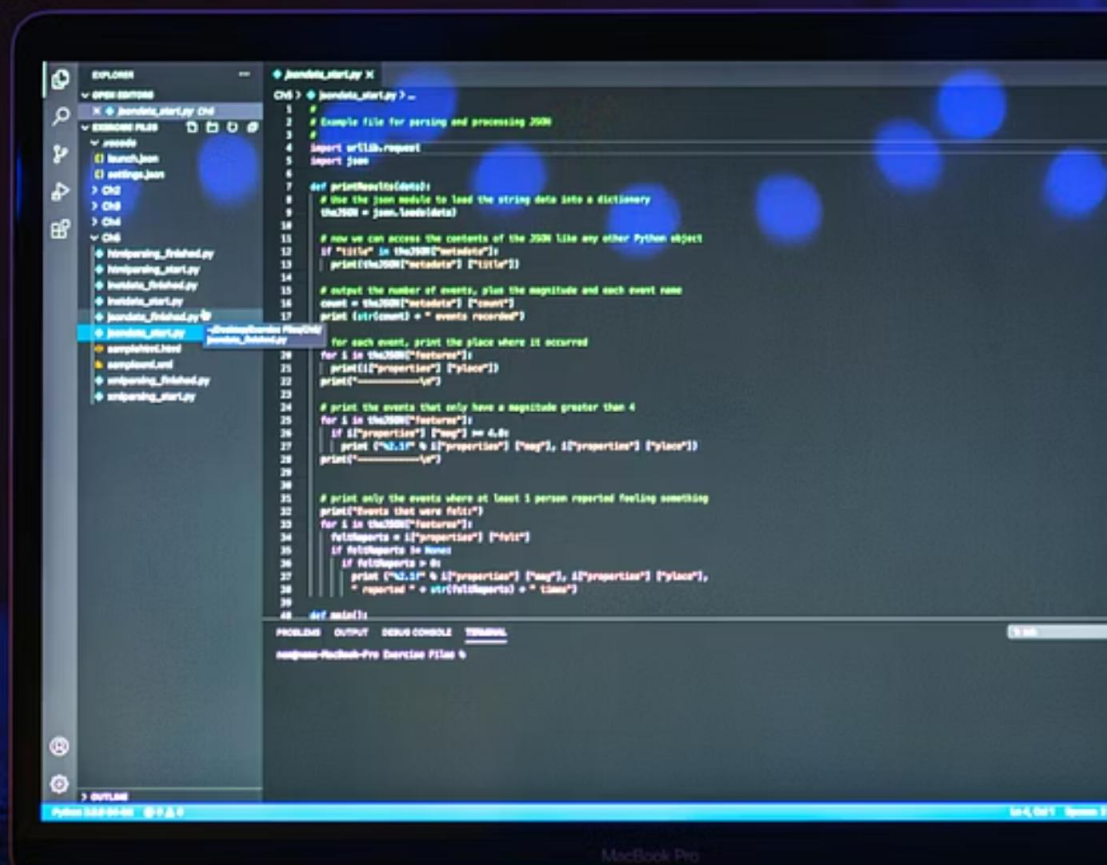


Interação com APIs

06

2026

SENAI - CRTI



Atualização e Exclusão com APIs

Concluindo o ciclo CRUD no Angular

Situação de Aprendizagem

O Contexto

Uma empresa possui um sistema de gerenciamento de tarefas já integrado com API. O time front-end agora precisa implementar as operações de modificação e remoção de dados.

O Desafio

- Editar tarefa existente
- Alterar status e marcar como concluída
- Excluir registro do servidor
- Atualizar a lista após cada operação



Pergunta-problema: Depois que um registro é criado, como o front-end consegue alterá-lo ou removê-lo corretamente no servidor?

CRUD de Tarefas

Título da tarefa

Ex.: Estudar Angular Signals

Descrição

Descreva a tarefa...

Salvar tarefa

Por que Atualização e Exclusão São Essenciais?

Aplicações reais vivem de **atualização constante de dados**. Qualquer sistema produtivo exige que o front-end comunique mudanças ao servidor de forma precisa e confiável.



Editar Cadastro

Corrigir nome, e-mail ou dados de um cliente já registrado.



Atualizar Status

Mudar um pedido de "pendente" para "em andamento" ou "entregue".



Alterar Perfil

Modificar foto, senha ou preferências de um usuário autenticado.



Cancelar / Excluir

Remover solicitações, cancelar pedidos ou inativar registros.

Revisão: O Ciclo CRUD Completo

O CRUD representa as quatro operações fundamentais de qualquer sistema que persiste dados. Nas aulas anteriores, implementamos **Create** e **Read**. Hoje completamos o ciclo.

Operação	Verbo HTTP	Ação	Exemplo de Rota
Create	POST	Criar novo recurso	POST /tarefas
Read	GET	Consultar recurso(s)	GET /tarefas
Update	PUT / PATCH	Atualizar recurso	PUT /tarefas/10
Delete	DELETE	Remover recurso	DELETE /tarefas/10

👉 **Meta da aula:** Implementar PUT, PATCH e DELETE para completar o ciclo CRUD no Angular com HttpClient.

PUT — Substituição Completa do Recurso

O que é PUT?

O método PUT realiza a **substituição completa** do recurso no servidor. Toda a representação anterior é descartada e substituída pelo payload enviado. Se um campo não for enviado, ele poderá ser perdido.

Payload obrigatório: todos os campos

```
PUT /tarefas/10
```

```
{  
  "titulo": "Estudar Angular",  
  "descricao": "CRUD completo",  
  "prioridade": "alta",  
  "categoria": "estudo",  
  "status": "concluida"  
}
```

Envia o recurso inteiro

Todos os campos devem estar presentes no corpo da requisição.

Substitui a representação anterior

O servidor descarta os dados antigos e grava os novos na íntegra.

Idempotente

Enviar o mesmo PUT múltiplas vezes produz o mesmo resultado.

Neste projeto, PUT foi implementado no service para completar o CRUD, mas a interface usa PATCH por ser mais eficiente para alterações pontuais.

PATCH — Alteração Parcial do Recurso

O que é PATCH?

O método PATCH realiza uma **alteração parcial** do recurso. Você envia apenas os campos que precisam ser modificados. Os demais campos do registro permanecem inalterados no servidor.

 PATCH altera **apenas o necessário** — ideal para mudar um único campo como o status de uma tarefa.

Payload mínimo: só o que muda

```
PATCH /tarefas/10

{
  "status": "concluida"
}
```

Apenas o campo `status` é enviado. Os outros campos do recurso não são alterados.

PUT

Envia **todos** os campos. Substitui o recurso inteiro.

PATCH

Envia **apenas os campos** alterados. Preserva os demais.

VERBO HTTP

DELETE — Remoção do Recurso

Sintaxe e Resposta

```
DELETE /tarefas/10
```

Status de resposta mais comum:
204 No Content — indica que a operação foi bem-sucedida e não há corpo de resposta.

Tipos de Exclusão

Exclusão Física

O registro é removido definitivamente do banco de dados.

Exclusão Lógica (Soft Delete)

O registro recebe uma flag como **ativo: false** e é ocultado, mas não apagado.

{RESTful API}

GET

POST

PUT

PATCH

DELETE

Service Angular: Implementando PUT, PATCH e DELETE

No Angular, os métodos HTTP são encapsulados no **service**, que utiliza o `HttpClient` para comunicação com a API. O endpoint usa template literal para inserir o `id` dinamicamente.

```
atualizar(  
  id: number,  
  tarefa: Tarefa  
) {  
  return this.http.put<Tarefa>(  
    `${this.api}/${id}`,  
    tarefa  
  );  
}  
  
atualizarParcial(  
  id: number,  
  dados: Partial<Tarefa>  
) {  
  return this.http.patch<Tarefa>(  
    `${this.api}/${id}`,  
    dados  
  );  
}  
  
remover(id: number) {  
  return this.http.delete(  
    `${this.api}/${id}`  
  );  
}
```

1

Endpoint dinâmico

Template literal `${id}` constrói a URL com o identificador do recurso.

2

Tipagem com Partial

`Partial<Tarefa>` torna todos os campos opcionais, perfeito para PATCH.

3

Observable retornado

Todos os métodos retornam um Observable que deve ser subscrito no componente.

Boas Práticas: Confirmação Antes de Excluir

Por que confirmar antes de excluir?

A exclusão acidental é um dos erros mais frustrantes para o usuário. Adotar boas práticas de confirmação protege a integridade dos dados e aumenta a confiança no sistema.

- Modal de confirmação
"Tem certeza que deseja excluir esta tarefa?"
- Opção de desfazer
Toast com "Desfazer" por alguns segundos após a exclusão.
- Alertas contextuais
Informar consequências antes da ação definitiva.

Reflexão de Design

Excluir **permanentemente** ou **arquivar**?

Soft delete preserva o histórico e permite recuperação. A exclusão física libera espaço, mas é irreversível. A escolha depende das regras de negócio.

```
excluir(id?: number) {  
  if (!id) return;  
  
  if (!confirm('Excluir  
tarefa?')) return;  
}
```



JSONPlaceholder: Testando as Operações

O **JSONPlaceholder** é uma API REST pública e gratuita, ideal para testes. As respostas simulam comportamento real, mas não persistem dados no servidor — perfeito para experimentar PUT, PATCH e DELETE durante o aprendizado.

Atualizar (PUT/PATCH)

```
https://jsonplaceholder.typicode.com/posts/1
```

Envie o payload completo (PUT) ou parcial (PATCH). A API retornará o objeto atualizado com status **200 OK**.

Remover (DELETE)

```
https://jsonplaceholder.typicode.com/posts/1
```

A API retorna **200 OK** com corpo vazio {}, simulando a exclusão do recurso.

Verbo	Recurso	Payload	Resposta esperada
PUT	/posts/1	Objeto completo	200 com objeto atualizado
PATCH	/posts/1	Campos parciais	200 com objeto mesclado
DELETE	/posts/1	Sem corpo	200 com {}

Atividade Guiada: Gerenciador de Tarefas CRUD

Mini Projeto

Implemente um gerenciador de tarefas completo consumindo a JSONPlaceholder. O projeto deve cobrir todas as operações do ciclo CRUD, com feedback visual ao usuário após cada ação.

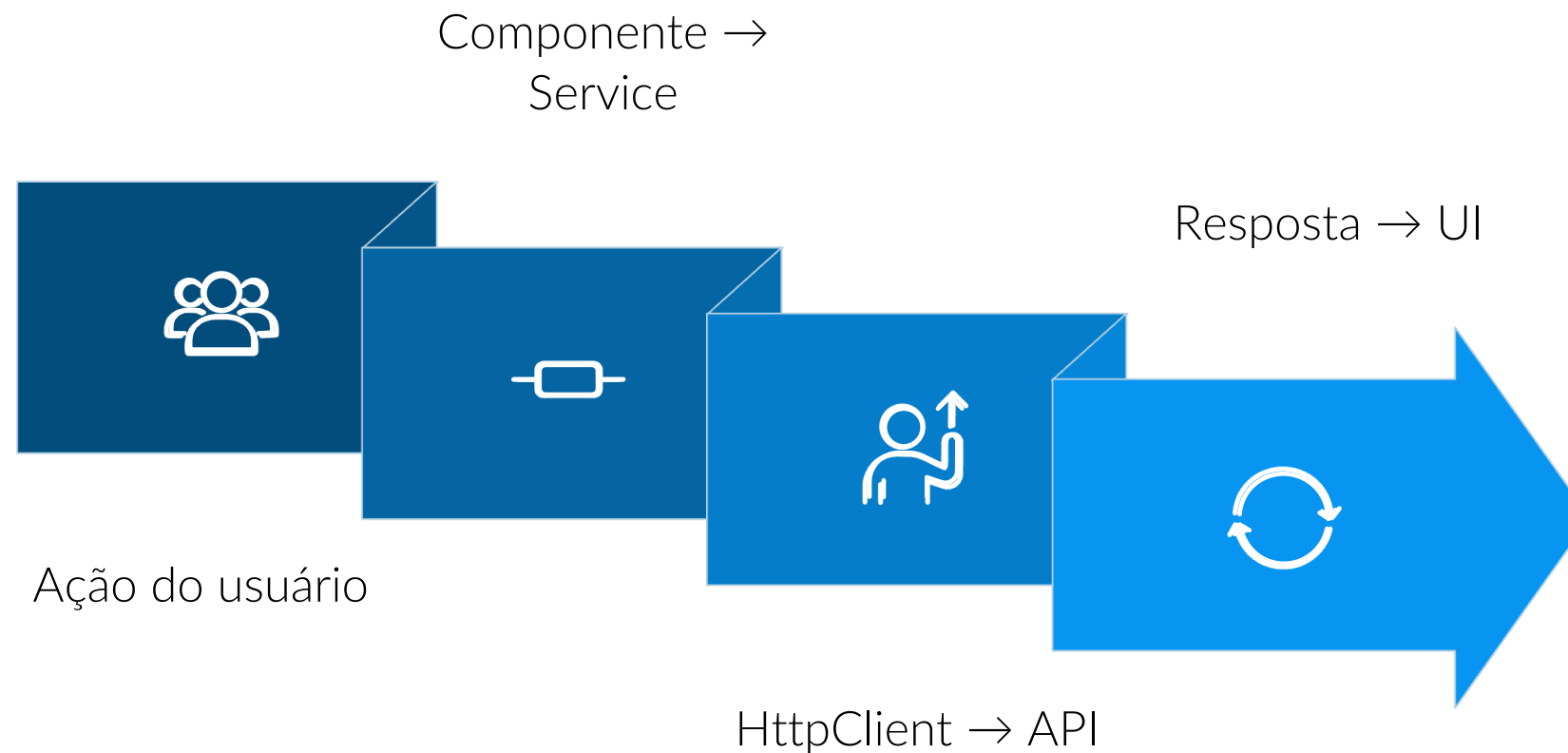
Checklist de Implementação

- ☐ Criar tarefa (POST)
- ☐ Listar tarefas (GET)
- ☐ Editar descrição (PATCH)
- ☐ Marcar tarefa como concluída (PATCH)
- ☐ Excluir tarefa (DELETE) com confirmação
- ☐ Atualizar UI e exibir feedback visual

- ☐ Utilize os métodos criados no service (`atualizar`, `atualizarParcial`, `remover`) e consuma-os nos componentes conforme demonstrado nesta aula.

Fluxo Completo: Front-end ↔ API

Entender o fluxo de comunicação entre o componente Angular e a API REST é fundamental para depurar erros e projetar sistemas robustos.



O Angular orquestra esse fluxo de forma reativa: o `HttpClient` emite um `Observable`, o `subscribe()` processa a resposta e o componente atualiza o estado local da interface — garantindo que servidor e tela permaneçam sempre sincronizados.

Reflexão Técnica: Tomada de Decisão no Back-end

Bons desenvolvedores não apenas implementam — eles **escolhem a abordagem certa** para cada cenário. Essas perguntas guiam decisões técnicas que impactam a qualidade do sistema.

Quando usar PUT?

Quando o formulário de edição contém **todos os campos** do recurso e você deseja garantir consistência completa na substituição.

Quando usar PATCH?

Quando apenas **um ou poucos campos** mudam — como marcar uma tarefa como concluída sem reenviar todos os dados.

Excluir ou inativar?

Se o registro tem **histórico ou auditoria**, prefira soft delete.
Se o dado não tem valor futuro, exclusão física é mais limpa.

Atualizar local ou buscar novamente?

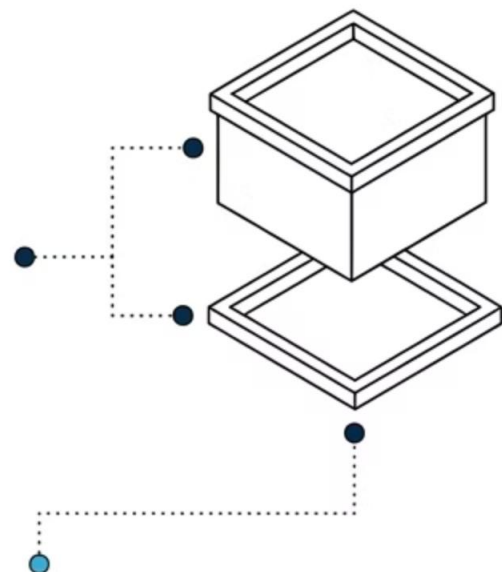
Atualizar localmente é mais rápido; buscar da API garante consistência com outros usuários. Avalie o contexto do sistema.

Comparativo: PUT vs PATCH

A escolha entre PUT e PATCH impacta diretamente o design da API e a eficiência da comunicação. Compreender as diferenças evita erros silenciosos e perda acidental de dados.

PUT

Substitui o recurso completo



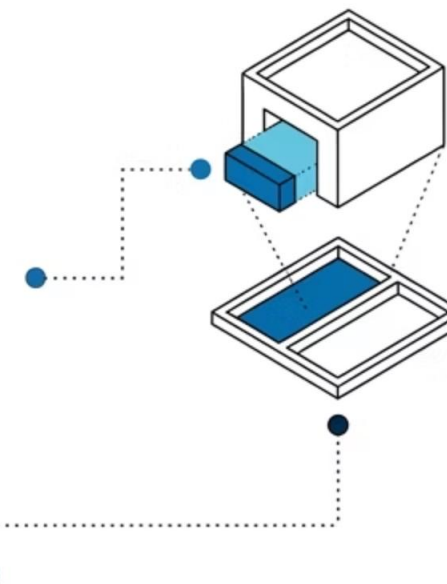
Envia todos os campos obrigatórios

Comportamento idempotente

Risco de perda de campos se omitidos

PATCH

Modifica parcialmente o recurso



Envia apenas campos alterados

Eficiente em banda

Ideal para atualizações pontuais como alterar status



Atenção: ao usar PUT, se você omitir um campo, o servidor pode salvar aquele campo como `null` ou vazio. Sempre envie todos os campos com PUT.

Sincronização: UI e Servidor

Estratégias de Atualização Local

Após uma operação bem-sucedida, há duas abordagens principais para manter a UI atualizada:

1 Atualização Otimista

Atualiza a UI imediatamente, antes da confirmação do servidor. Mais rápida, mas requer rollback em caso de erro.

2 Atualização Conservadora

Aguarda a resposta do servidor antes de atualizar a UI. Mais segura e consistente.



Evidência de Competência

Ao concluir esta aula e entregar a atividade proposta, o aluno demonstra domínio das competências técnicas previstas no plano de ensino desta disciplina.



Implementar PUT

Substituição completa de um recurso via requisição HTTP com payload integral.



Implementar PATCH

Atualização parcial enviando apenas os campos modificados ao servidor.



Implementar DELETE

Remoção de recursos com tratamento de resposta e atualização da



Diferenciar Atualização

Escolher entre PUT e PATCH conforme o contexto e os requisitos do sistema.



Sincronizar UI

Manter a interface do usuário consistente com o estado real dos dados no servidor.



CRUD Completo

Dominar o ciclo completo Create, Read, Update e Delete em Angular com HttpClient.

